

Redaction Recovery: Featherweight Pseudocode

Corpus Loading — Wikipedia XML Dump Processing

Weight Class: Beginner

Sub-Stage: Featherweight (1a)

Component: Load Wikipedia Corpus

Idiom: Generator-based corpus load with token count filtering

Locked Config: `mediawiki` package, word-split tokenization, 500–5000 token filter

Stage 1: Pseudocode (Language-Agnostic)

Input

- File path to Wikipedia XML dump (e.g., `enwiki-latest-pages-articles-multistream.xml.bz2`)

Output

- Stream of (doc_id, raw_text) tuples
 - Each tuple represents one article that passed filtering
-

Block 1: Open and Iterate Through Wikipedia Dump

```
FUNCTION load_wikipedia_dump(dump_path):  
    // Opens the Wikipedia XML dump file  
    // Handles .xml, .xml.bz2, .xml.gz automatically (based on file extension)  
  
    dump_stream = OPEN_WIKIPEDIA(dump_path)  
  
    FOR EACH page IN dump_stream:  
        YIELD page
```

Decision Points Named:

- File handling: Auto-detect compression (`.bz2`, `.gz`, or uncompressed)
 - Parser: Use `mediawiki` package (handles Wikipedia XML schema)
 - Iteration: Lazy evaluation (generator, not list in memory)
-

Block 2: Extract Title and Plaintext

```
FUNCTION extract_page_content(page):  
    // From a page object, extract title and plaintext  
  
    title = page.title
```

```
raw_text = page.text

RETURN (title, raw_text)
```

Decision Points Named:

- Title source: `page.title` (the article name, e.g., "Napoleon")
 - Plaintext source: `page.text` (already stripped of markup by `mediawiki` package)
 - No manual cleanup needed (`mediawiki` handles it)
-

Block 3: Create Deterministic Document ID

```
FUNCTION create_doc_id(title):
    // Convert article title to a stable, unique identifier

    doc_id = "wikipedia_" + SLUGIFY(title)

    RETURN doc_id
```

Decision Points Named:

- Prefix: `"wikipedia_"` (namespace, for clarity in mixed datasets)
 - Slugify: Convert to lowercase, replace spaces/punctuation with underscores
 - Example: `"Napoleon Bonaparte"` → `"wikipedia_napoleon_bonaparte"`
 - Deterministic: same title always produces same `doc_id`
-

Block 4: Count Tokens (Word-Split Approximation)

```
FUNCTION count_tokens_approximate(raw_text):
    // Quick token count using whitespace split
    // Not exact (punctuation not separated), but O(1) and deterministic

    tokens = SPLIT(raw_text, whitespace)
    token_count = LENGTH(tokens)

    RETURN token_count
```

Decision Points Named:

- Tokenization: Whitespace split (simple, fast, reproducible)
 - No regex, no NLP model (that comes in Lightweight stage)
 - Edge case: Multiple consecutive spaces count as one split
-

Block 5: Filter by Token Count

```

FUNCTION should_keep_article(raw_text):
    // Determine if article should be kept based on token count

    token_count = count_tokens_approximate(raw_text)

    IF token_count < 500:
        RETURN FALSE

    IF token_count > 5000:
        RETURN FALSE

    RETURN TRUE

```

Decision Points Named:

- Min threshold: 500 tokens (enough context for entity recovery task)
 - Max threshold: 5000 tokens (fits in memory, reasonable article length)
 - Token count: Approximate (word-split), not exact
-

Block 6: Generator Pipeline (Compose All Blocks)

```

FUNCTION corpus_loader(dump_path):
    // Main generator: open dump, extract, filter, yield

    dump_stream = load_wikipedia_dump(dump_path)

    FOR EACH page IN dump_stream:
        (title, raw_text) = extract_page_content(page)

        IF NOT should_keep_article(raw_text):
            SKIP

        doc_id = create_doc_id(title)

        YIELD (doc_id, raw_text)

```

Decision Points Named:

- Order: Filter *before* yielding (no wasted memory)
 - Generator semantics: Lazy evaluation (one article at a time, not all in memory)
 - Error handling: Skip articles with extraction errors (malformed XML), don't crash
-

Stage 2: Python Implementation (Locked Idiom)

You will implement using:

- `mediawiki` package (`mediawiki.MediaWiki()` or `mediawiki.MediaWikiDump()`)

- Generator functions (`def corpus_loader(...): yield ...`)
- `.split()` for word tokenization
- `len()` for token count
- String `.replace()` or similar for slugify

Constraints:

- No `pandas`, use `polars`
- No async, no multiprocessing, no file buffering optimizations
- One article at a time

Syntax help only. Idiom is locked.

Stage 3: Validation

Test against these constraints:

1. **Corpus loads without error:** Run generator, count articles yielded
 - Expected: 500–1000 articles (adjust dump size as needed)
2. **Token count filter works:** Manually check 5 articles
 - Verify: All yielded articles have 500–5000 tokens (word-split count)
 - Verify: At least one article was skipped (< 500 or > 5000 tokens)
3. **doc_id is deterministic:** Run generator twice on same dump
 - Verify: Same articles yield same doc_id (lowercase, slugified title)
4. **raw_text is plaintext:** Spot-check 3 articles
 - Verify: No XML markup, no `<ref>`, no `{{template}}` syntax
 - Verify: Readable prose (spaces between words)
5. **Token count approximation vs. actual length:** Sample 10 articles
 - Verify: `len(raw_text.split())` matches your token count

Pass criteria: All 5 checks pass. Then: **Next sub-stage (Lightweight).**

Appendix: Locked Configuration

Parameter	Value	Reason
Dump format	Wikipedia XML (.bz2, .gz, or uncompressed)	Standard dump format
Parser	<code>mediawiki</code> package	Handles Wikipedia schema, stripping auto
Title source	<code>page.title</code>	Article name

Parameter	Value	Reason
Plaintext source	<code>page.text</code>	Already stripped by mediawiki
doc_id prefix	<code>"wikipedia_"</code>	Namespace clarity
doc_id slugify	Lowercase + underscore-replace	Deterministic, filesystem-safe
Token count	Word-split approximation (<code>split()</code> on whitespace)	O(1), reproducible
Min article length	500 tokens	Enough context
Max article length	5000 tokens	Memory-friendly
Iteration style	Generator (yield)	Lazy evaluation, no memory bloat